

QA AUTOMATION FRAMEWORK GUIDE

Playwright • Selenium • Cypress • PyTest • CI/CD

Brian Scott McCarthy | Senior QA Lead | github.com/BrianGator

1. Framework Overview

This guide documents the automation testing frameworks designed and maintained by Brian Scott McCarthy across multiple enterprise projects. All frameworks follow industry best practices including Page Object Model (POM), data-driven testing, CI/CD integration, and Azure DevOps reporting.

Framework	Language	Scope	CI/CD	GitHub Repo
Playwright	TypeScript	E2E, Regression, Cross-browser, Mobile	GitHub Actions	Playwright-Test-Automation-2-w-TypeScript
Selenium WebDriver	Java	UI Regression, Multi-env, Grid	Jenkins	Selenium-Test-Automation2-w-Java
Cypress	JavaScript	Component, Integration, API intercept	GitHub Actions	Cypress-Automation-Testing-w-Javascript
PyTest	Python	API validation, DB checks, scripted data tests	GitHub Actions	Python-PyTest-Scripting
.NET / C# Tests	C#	File processing, unit + integration	Azure DevOps Pipelines	File-Processing-App-w-CS-.NET-Tests
ASPX / C# Tests	C# / HTML	Media Library web app testing	Azure DevOps Pipelines	Media-Library-w-CS-.NET-ASPX-Tests
React / JS Tests	JavaScript	e-Plant Shop web app, component tests	GitHub Actions	e-Plant-Shop-Web-App-w-JavaScript-React-Tests
Travel Site Tests	JavaScript	Search, filter, UI tests	GitHub Actions	Travel-Website-w-Search-JavaScript-HTML-CSS-Tests

2. Playwright Framework (TypeScript) — Primary

2.1 Architecture

- Pattern: Page Object Model (POM) — one class per page; methods encapsulate interactions.
- Test Runner: Playwright Test (@playwright/test) with built-in assertions.
- Parallelism: Playwright workers run tests in parallel across browser projects.
- Configuration: playwright.config.ts defines projects (Chromium, Firefox, WebKit), baseURL, retries, and reporters.

2.2 Folder Structure

Path	Contents
tests/	All test spec files (.spec.ts) organised by feature module
pages/	Page Object classes (e.g. LoginPage.ts, CheckoutPage.ts, SearchPage.ts)
fixtures/	Custom Playwright fixtures for shared setup (auth state, test data)
test-data/	JSON files for data-driven tests (users, products, addresses)
utils/	Helper functions — API helpers, DB query wrappers, date utilities
reports/	HTML reports, screenshots, trace files (auto-generated per run)
playwright.config.ts	Browser projects, timeout settings, reporter config, base URL
.env	Environment-specific variables (TEST_URL, TEST_USER, DB_CONN)

2.3 Key Capabilities

- Auto-waiting eliminates flaky sleep() calls; assertions retry until condition met or timeout.
- Network interception for API mocking and response validation without UI dependency.
- Visual regression via screenshot comparison on critical UI components.
- Trace viewer for debugging failed CI runs — step-by-step DOM snapshots.
- Playwright Codegen used to accelerate new test authoring.

2.4 CI/CD Integration — GitHub Actions

Trigger	Workflow	Browsers	Report Destination
Pull Request	Run affected test suites (smoke + changed module)	Chromium	GitHub Actions summary + Azure DevOps
Merge to main	Full regression suite	Chromium, Firefox, WebKit	HTML report + ADO pipeline
Nightly (00:00 UTC)	Full suite + performance smoke	All 3 browsers	Azure DevOps nightly run

3. Selenium WebDriver (Java) — Legacy Suite

- TestNG test runner with @Test, @BeforeMethod, @AfterMethod annotations.
- Maven build system; pom.xml manages all dependencies (selenium-java, testng, REST Assured).
- Selenium Grid enables parallel execution across 8 application environments simultaneously.
- WebDriverManager handles driver binary management (no manual chromedriver setup).
- Extent Reports generates HTML test reports published to Jenkins build dashboard.
- Jenkins pipeline triggers nightly; results published to Azure DevOps via REST API.

4. Cypress (JavaScript) — Integration Layer

- Ideal for component-level tests and scenarios requiring network request interception.

- `cy.intercept()` stubs external API calls for fast, isolated UI validation.
- Cypress Dashboard (cloud) stores video recordings and screenshots of CI runs.
- Command: `npx cypress run --spec 'cypress/e2e/**/*.cy.js' --browser chrome`
- Repo: github.com/BrianGator/Cypress-Automation-Testing-w-Javascript (MIT License)

5. Python PyTest — API & Data Validation

- Lightweight scripts for REST API assertions, JSON schema validation, and SQL data-integrity checks.
- `pytest-html` generates HTML reports; `allure-pytest` generates Allure reports for stakeholders.
- Fixtures (`@pytest.fixture`) manage test data setup and teardown.
- Used post-deployment to validate database state matches expected records after transactions.
- Repo: github.com/BrianGator/Python-PyTest-Scripting

6. Azure DevOps Integration

Integration Point	Details
Test Plans	All test suites created in ADO Test Plans; manual cases executed and tracked within ADO.
Pipeline Triggers	Playwright and Selenium pipelines defined as YAML pipelines in Azure DevOps; triggered on PR and schedule.
Test Result Publishing	JUnit XML results published to ADO pipeline using 'Publish Test Results' task; visible in test run dashboard.
Defect Logging	Failed test cases link directly to ADO Bug work items; fields auto-populated from test run context.
Traceability	Test cases linked to User Stories via ADO requirement-based test suites; coverage matrix auto-generated.
Reporting	Sprint test summary exported from ADO as HTML/PDF; metrics included in sprint review presentation.

7. Best Practices & Standards

Test Authoring Standards

1. Every test case must have a single, clearly stated assertion goal — no multi-purpose tests.
2. Test data must be externalised (JSON/env vars) — never hardcoded in test files.
3. Page Object methods must return page objects (fluent API) to enable readable chaining.
4. Failed tests must include a screenshot and, for Playwright, a trace file for CI debugging.
5. Every new automated test case must be linked to its ADO test case ID in a `@Tags` annotation or describe block comment.

Naming Conventions

Artifact	Convention	Example
Spec file	[feature].spec.ts	checkout.spec.ts
Page Object	[Page]Page.ts	CheckoutPage.ts
Test describe block	[Module] — [Sub-feature]	Checkout — Payment Processing
Test it/test name	should [expected behaviour] when [condition]	should place order successfully when valid card provided
ADO Test Case	TC-[#####]-[ShortTitle]	TC-0021-CheckoutValidCard

8. GitHub Repository Summary

Repository	Language	Description
Playwright-Test-Automation-2-w-TypeScript	TypeScript	Primary E2E framework — POM, cross-browser, CI/CD integrated
Selenium-Test-Automation2-w-Java	Java	Legacy regression suite — TestNG, Maven, Selenium Grid
Cypress-Automation-Testing-w-Javascript	JavaScript	Component & integration testing — MIT Licensed framework
Python-PyTest-Scripting	Python	API validation & DB integrity scripts
File-Processing-App-w-CS-.NET-Tests	C#	File processing app with .NET unit & integration tests
Media-Library-w-CS-.NET-ASPX-Tests	C# / HTML	Media library ASPX web app with test coverage
e-Plant-Shop-Web-App-w-JavaScript-React-Tests	JavaScript	React e-commerce app with JavaScript test suite
Travel-Website-w-Search-JavaScript-HTML-CSS-Tests	JavaScript	Travel site with search functionality and full test coverage
Selenium-Test-Automation-w-Java	Java	Selenium WebDriver automation — foundational Java framework